LedgerLens — Detailed Business Proposal

Accenture Innovation Challenge 2026 · Round 2 · Track 3, BusinessIntelligence.ai

A KPI intelligence-to-action engine. Root-cause analysis as a set intersection over a ledger of business changes, verified by negative controls, narrated by an AI investigator that proposes but never decides.

Repository: https://github.com/omega-sus67/LedgerLens · **343 passing tests** · runs end to end with no API key

---

Executive summary

Dashboards report *what* moved. Establishing *why* takes an analyst days across Slack, Jira, deploy logs and Zendesk — and the expensive failure is not the delay, it is acting on the wrong why.

LedgerLens takes an anomalous KPI, narrows it to the cohort that actually moved, intersects that cohort with every recorded change in the business, and then **tries to kill each candidate** with automatically generated negative controls. What survives is ranked, and every number on screen links to the SQL that produced it.

On the reference incident it diagnoses a **−$416,144** renewals shortfall in **1.3 seconds**, and — the point of the whole design — it **rejects the plausible-but-wrong explanation** that a human under time pressure would likely have acted on.

Time to a defensible diagnosis	**~3 analyst-days → 1.3 seconds**
Numbers a reader can replay	**22 distinct `query_id`s** on one card
Documented value per customer, per year	**~$205,000** — recovered analyst capacity plus one avoided misattribution
Cost to serve one customer, per year	**under $1** in model spend; the engine runs on the client's own warehouse
LLM cost per diagnosis	**~$0.0048** (Gemini Flash), **$0.0000** with the AI lane off

The commercial shape follows from the technical one. Because the ranking path is deterministic SQL rather than inference, marginal cost is close to zero and does not grow with usage — the opposite of a design that calls a model for every question. At a licence priced near a quarter of documented value, the customer sees a **4× first-year return** on a number they can audit themselves, and the margin behaves like classic software rather than like AI tooling. Sections 4.5–4.8 set out the unit economics, pricing, adoption scenarios and what compounds.

---

1. Problem framing

The observable problem

A dashboard reports that renewals in DACH fell 8%. It cannot say why. The question goes to an analyst, who spends the next three days reconstructing a narrative from Slack threads, deploy logs, support tickets and vendor emails — and by the time the answer arrives, the decision window has closed.

The expensive problem

The delay is not the costly part. Acting on the wrong "why" is.

In any given week, dozens of deliberate changes coincide with any metric movement. In our reference incident, marketing spend was cut in DACH **one day before** renewals broke. That correlation is real, it is temporally *more* convenient than the truth, and acting on it — restoring the campaign — costs money and fixes nothing. The actual cause was a payment-connector release that silently broke SEPA direct-debit renewals for enterprise customers in that region.

A tool that ranks by plausibility confidently recommends the wrong action here. That is the failure mode this system is built to eliminate.

Why this is hard, in the brief's own terms

Real-world complexity	Why naive approaches break
Multiple interacting drivers — price, mix, marketing, seasonality, competition	Correlation ranks the decoy first
Different grains, refresh cadences, historical coverage	A single global threshold makes a 98.2% rate KPI undetectable in principle
Sparse history for new products	Detection has nothing to fit a baseline against
Contradictory evidence, confidence calibration	A confident wrong answer is worse than no answer
Role-based security and sensitive dimensions	The same question must return different cuts to different readers
LLM economics — model choice, tokens, latency, cost	An LLM in the scoring loop is unbounded cost *and* unbounded risk

Why now

Blast radius only became queryable in the last five years. GitOps, feature-flags-as-a- service, campaign platforms with APIs and infrastructure-as-code turned deliberate change into machine-readable metadata with *declared* targeting: a deploy records its rollout regions, a flag records its targeting rules, a campaign records its geo. A decade ago that linkage would have had to be **inferred** from prose — which is precisely the design this system rejects, because an inferred link cannot be audited.

The second enabler is LLM economics. At roughly half a cent per diagnosis, a model is cheap enough to *propose* additional checks and rewrite prose for a given reader without ever being trusted with the verdict. The "propose but never decide" split is affordable now in a way it was not when a single call cost dollars and the only way to justify it was to put it on the critical path.

The claim we deliberately do not make

This does not prove causation, and it does not try to. With a single incident there is no population over which to estimate an effect, so any tool claiming "causal inference" here is overselling. What LedgerLens delivers is *ranked, auditable evidence plus the test that would settle it* — which is what an analyst actually needs in order to act.

---

2. Solution design

2.1 The organising principle

> **The LLM should not be treated as the source of quantitative truth. Teams should explicitly demonstrate when they use deterministic logic, SQL, statistics, business rules, traditional ML, causal inference, retrieval or LLMs — and why.** — Round 2 brief, Track 3

This is the sentence the architecture is built around. Our answer is a **deterministic spine with probabilistic edges**:

Capability	Implemented with	Why that choice
Anomaly detection	Robust statistics — MAD z-score on rolling-median residual, STL when enough cycles exist	Outlier-resistant; needs no training data
Attribution / drill-down	Contribution analysis over a dimension lattice, top-down with a contribution floor	Bounds tests to the anomalous path, not the full cross-product
Multiple-testing control	Benjamini–Hochberg FDR	Labels, never gates
Context linkage	**Change Ledger** — typed events, each with a machine-readable *blast radius*	90% of enterprise linkage is already recorded. Foreign keys beat LLM entity extraction on cost, latency and error rate
Candidate generation	**SQL set intersection** of blast radius against the affected cohort	Sub-millisecond, explainable in one sentence
Ranking	Weighted rubric of five named components, each a query	A confidence number a judge cannot audit is a number that costs you points
Falsification	**Automatically generated negative controls**	The N=1-appropriate replacement for a causal-inference library's refuters
Learning	Beta–Bernoulli posterior counted from analyst verdicts	No model state to drift; deleting a verdict reverses it exactly
Proposing further checks	**LLM**, constrained to a template vocabulary	The rules cover the checks you anticipated; the LLM covers the ones you didn't
Naming untestable causes	**LLM**, in a separately labelled panel	The honest answer to "what if the cause was never recorded?"
Narration	**LLM**, behind a numbers guard	Prose is a rendering concern; numbers are not

2.2 The core primitive: blast radius

Every deliberate business change is recorded as a typed `ChangeEvent` carrying the exact slice of customers it *could* have touched:

```
deploy_sepa_v214   blast radius: {region: [DACH], payment_rail: [sepa]}
campaign_dach_cut  blast radius: {region: [DACH]}
```

"Is this change relevant to this anomaly?" becomes a **set intersection**, not a graph traversal or an embedding-similarity search. Deterministic, sub-millisecond, and explainable to a non-technical stakeholder in one sentence.

This is why the design needs no graph database, no vector store and no agent framework.

2.3 Scoring: five components, every one a query

```
score = 0.25·T + 0.30·C + 0.15·D + 0.25·N + 0.05·P
```

	Measures	Explicitly does not mean
T temporal	The change began shortly before the metric broke	Precedence is necessary for causation, never sufficient
C cohort match	Row-level Jaccard between blast radius and affected cohort	A wide radius scores badly even if it *was* the cause
D dose–response	Rank correlation of exposure against impact	Uninformative for this incident, and the card says so rather than manufacturing a number
N negative controls	Fraction of falsifiable predictions that survived	A passing control is a *failure to falsify*, not a confirmation
P learned prior	Beta–Bernoulli mean over past analyst verdicts	Weighted 0.05; sharpens a ranking, never decides one

2.4 The mechanism that matters: killing the decoy

For each candidate the engine constructs the cohorts its blast radius says should be **unaffected**, and the cohorts a rival mechanism says should **also** have moved, then checks whether reality agrees.

The marketing cut is a demand-side change to a *region*. If it caused the drop, DACH Mid-market and SMB customers should have fallen too. They did not — they came in **flat at −1.3%**. That control failure is decisive, and the decoy is **rejected outright at 0.322**, not merely outranked.

Candidate	T	C	D	N	Total	Outcome
`deploy_sepa_v214`	1.00	0.33	0.50	**1.00**	**0.700**	ranked #1
`campaign_dach_cut`	0.72	0.14	0.50	**0.00**	**0.322**	**REJECTED**

Being second is not good enough. A decoy still on the list is a decoy an executive might act on.

2.5 The AI investigator lane

Three LLM call sites, all **additive** to the deterministic spine:

1. **Proposed checks** — the model reads the anomaly, the ranked candidates and the controls already run, then proposes up to six further checks by filling a **fixed template vocabulary**. Our engine executes them in SQL. In the reference run it proposed a support-ticket check that **failed at +2766.7%** — a ticket spike that falsifies "this cohort was unaffected", which is evidence *for* the leading candidate.

2. **Unverifiable causes** — a separately labelled panel of explanations the connected data cannot test, each naming the feed that would settle it. The honest answer to *what if the real cause was never recorded?*
3. **Guarded narration** — persona-voiced prose, where every numeric token is checked against the verified payload. **One invented digit discards the narration** in favour of the deterministic template, and the page says so when it happens.

Nothing the model returns can change a rank. Enforced in code, not by convention: proposed checks are constructed with `decisive=False` — the field the N-scorer reads — and are never passed to it. A test asserts every score is byte-identical with the lane on and off.

Guards report what they caught. Hallucinated dimension values and attempts to escape the template vocabulary are rejected *before* becoming a query, and the count is displayed ("4 accepted, 2 rejected by validation"). A validator that never reports catching anything is indistinguishable from one that is not running.

2.6 Provider-agnostic by construction

One table of provider rows and one adapter class each. **Gemini 2.5 Flash by default** over hand-rolled REST; **Claude Sonnet** via the official SDK using forced tool-use. The two share no code path and no wire format, and the investigator cannot tell them apart.

```
LEDGERLENS_LLM_PROVIDER=anthropic   # switch vendor, no source change
LEDGERLENS_LLM_MODEL=gemini-2.5-pro # switch model within a vendor
```

The `Provider` interface has **exactly one method**, and a test keeps it that way — a second method would necessarily describe one vendor's feature, and vendor-specific branching would follow.

2.7 Governance, security and honesty

- **KPI semantic contract** — definition, calculation SQL, drivers, thresholds, lineage, measured freshness and access rules, read by the engine and rendered in the UI. Not decorative metadata: the thresholds in force are the contract's.
- **Role-based entitlement** — enforced at exactly one chokepoint. Growth marketing loses the `payment_rail` cut, the card **names the policy** that withheld it, and the numbers legitimately differ because the focal cohort differs.
- **Abstention** — when no candidate clears the score floor, the system reports exactly that, lists which sources are and are not connected, and recommends widening ingestion. The failure direction is the design's saving property: **a too-wide blast radius fails its controls; a too-narrow one leaves nothing above the floor.** It degrades toward *I don't know*, never toward a confident wrong answer.
- **Provenance** — one function is the only path to the database. It hashes SQL plus parameters into a `query_id` and logs it, so every number on a card is replayable. There are exactly **two** deliberate exceptions, both facts about the *process* rather than the data — runtime telemetry and policy redactions — and both say so on screen rather than leaving a gap to be noticed.

2.8 How this differs from what already exists

Category	What it does	Where it stops
Dashboards — Power BI, Tableau, Looker	Report *what* moved	No *why* at all
Anomaly detection — Anodot, Monte Carlo	Detect that something moved, and alert	Detects; never explains
Augmented analytics — ThoughtSpot SpotIQ, Sisu	Rank contributing drivers statistically **within the warehouse**	Only sees columns. **A deploy is not a column.**
LLM copilots — Databricks Genie, Power BI Copilot	Natural-language Q&A over the data	The model *composes* the answer, so a confidently wrong one is indistinguishable from a right one — and neither is auditable
LedgerLens	Intersects the anomaly with a **ledger of business changes**, then tries to *falsify* each candidate	Does not prove causation — and says so on the card

The distinction is not incremental:

> **The cause of a metric movement is almost never in the metric's own table.** Every tool above searches the *data* for the answer. LedgerLens searches the **change log** — and then tries to disprove what it finds.

Two consequences follow, and both are structural rather than a matter of tuning. Because candidates come from a change ledger rather than from column statistics, a cause with no statistical signature in the warehouse — a connector release, a flag flip — is still findable. And because ranking is by *survived falsification* rather than by correlation strength, the plausible-but-wrong candidate is **rejected outright** rather than merely placed second. A decoy still on the list is one an executive might act on.

---

3. Target users

Four personas render from **one computation**. Persona is accepted only by the narrator, which sits downstream of every query — so it *cannot* reach a query, and the evidence behind all four cards is identical by construction.

Persona	Channel	Depth	Decision rights	What they get
Revenue Analyst	workspace	full	all levers	Every control, every query id, the drill-down lattice
CFO	email digest	summary	`hold_forecast`	Dollars, forecast risk, an escalation — never an instruction to roll back a release
Payments On-Call	pager	operational	`rollback_release`, `disable_flag`	The event id, the blast radius, the rollback
Growth Marketing	workspace	summary	`restore_campaign_budget`	Their own KPI's real story — and a named policy where a cut is withheld

Decision rights are mechanical, not cosmetic. A persona that does not hold a lever is shown an *escalation*, never an instruction. A CFO is never told to roll back a release.

Recommendations follow the brief's chain exactly — **driver → controllable lever → action → expected impact → owner → confidence → monitoring plan** — plus a `basis` field carrying the `query_id`, so any recommendation opens onto the SQL underneath it.

---

4. Business case and impact

4.1 Stated assumptions

The brief invites reasonable assumptions, clearly stated. Ours, for a mid-market B2B SaaS business at roughly €50M ARR:

Assumption	Value	Basis
Material KPI movements needing investigation	~40 / year	Roughly weekly across 3–5 tracked KPIs
Analyst time per investigation	~3 days	The status quo the problem statement describes
Fully-loaded analyst cost	~$460 / day	~$120k total annual cost ÷ 260 days
Incidents where the first-guess cause is wrong	~1 in 4	The decoy pattern: a coincident change that is temporally more plausible

These are directional planning figures for sizing the opportunity, not measured results.

4.2 Where the value is

Analyst time recovered. 40 investigations × 3 days × $460 ≈ **$55k/year** of analyst capacity, redirected from evidence assembly to judgement. Real, but the smaller half.

Wrong-action avoidance — the larger half. In the reference incident the two costs of acting on the decoy compound:

- The **−$416,144** renewals leak stays open for however long the wrong fix is given to work.
- The campaign cut is reversed on a false premise, when the controls show it *did* cause a genuine ~31% new-logo drop — a real finding about a different metric.

One avoided misattribution per year on an incident of this size dominates the entire labour saving. This is why the negative-control layer, not the ranking, is the product.

Decision-window preservation. A diagnosis in **1.3 seconds** rather than three days means the finding lands while the incident is still live and the fix is still cheap.

4.3 What it costs to run

Diagnosis latency	**1,293 ms** cold; ~2.6× faster warm
Database work	89 registered queries executed, 41 served from cache
LLM cost per diagnosis	**~$0.0048** on Gemini 2.5 Flash (3 calls, ~6k in / 1.2k out)
LLM cost with the lane disabled	**$0.0000** — the ranking path never calls a model
Same diagnosis on Claude Sonnet	~$0.0240, no source change
Infrastructure	One DuckDB file. No graph DB, no vector DB, no agent framework, no orchestrator

At ~40 diagnoses/year the model spend is **under $1/year**. Cost is not the constraint; trust is — which is why the engineering went into falsification and provenance rather than into a larger model.

4.4 Why this generalises

The only thing that changes per customer is the **connector mapping** from existing metadata to blast radius. A deploy already knows its rollout regions, a feature flag knows its targeting rules, a campaign knows its geo, a ticket knows its account and the account knows its segment. **The linkage is already recorded; it does not need to be inferred.**

4.5 What it costs to serve — and why the margin is unusual

The measured figures in §4.3 have a commercial consequence worth stating plainly, because it is the most attractive property of this design and it was not designed for that reason.

Cost line	Per customer, per year	Why
Model spend	**under $1**	~40 diagnoses × $0.0048, and $0 with the lane off
Compute	**the customer's own**	The engine is SQL. It executes warehouse-native against Snowflake, Databricks or BigQuery — there is no second copy of the data and no cluster to run
Dedicated infrastructure	**none**	No graph database, no vector store, no agent orchestrator, no GPU
Support, updates, rule-library maintenance	the real cost	Human, and shared across every customer

Almost all marginal cost in this category is inference and infrastructure, and this design has close to neither. A competitor that puts an LLM on the ranking path pays for a model call on every question, at every scale, forever. We pay for one only when a reader asks for extra checks or nicer prose — and can turn it off entirely without losing a number.

The result is a gross margin that behaves like classic software rather than like AI tooling — and it *improves* with scale, because the expensive part (the negative-control rule library) is written once and shared by every deployment.

4.6 Pricing, anchored to measured value

Two revenue lines, because this lands as both an engagement and an asset.

1. Implementation. The connector mapping is the deliverable (§4.9) — client-specific configuration, not custom engineering. A 6–8 week shape for a first KPI family, with each additional source measured in days.

2. Annual platform licence, priced against documented value rather than seat count:

	Per customer, per year
Analyst capacity recovered (§4.2)	~$55,000
Expected value of misattribution avoided	conservatively ~$150,000 — one incident a year at a fraction of the reference incident's $416k
Documented annual value	**~$205,000**
Licence at 20–25% of value captured	**~$45,000 – $50,000**

That leaves the customer a **4× first-year return** on a number they can audit themselves, which is the easiest procurement conversation available. And because the value scales with the number of KPIs under management rather than with users, expansion inside an account is a configuration change rather than a new sale.

These are planning figures for sizing, not quoted prices. The value inputs are the §4.1 assumptions; the capture ratio is a standard enterprise-software heuristic.

4.7 What the opportunity looks like at scale

Rather than claim a share of a large market, here is the arithmetic at three adoption levels inside a systems integrator's existing client base. A qualifying client is one with a data warehouse, three or more tracked KPIs, and an analytics or revenue-operations function — which describes most subscription and transaction-driven businesses above roughly $20M revenue.

	Clients live	**Recurring licence ARR**	**Implementation, new clients that year**
Pilot year	10	~$0.5M	~$1.2M (10 × ~$120k)
Established	50	~$2.4M	~$2.4M (20 × ~$120k)
At scale	150	~$7.1M	~$3.6M (30 × ~$120k)

At the "established" line, recurring revenue passes one-off revenue — the point at which this stops being a service line and starts being an asset. The engineering required to get from 10 to 150 clients is connector coverage, not new science: the engine is unchanged, and every rule added for one client is available to all of them.

4.8 Why a follower cannot simply copy it

Three things compound, and none of them is the algorithm:

- **The negative-control rule library.** Each rule encodes a falsifiable mechanism assumption that a domain expert argued for. Five ship today; every incident investigated is a candidate for a sixth, and each one improves every deployment at once.
- **The verdict corpus is per-tenant and cumulative.** Analyst confirmations sharpen a client's own prior, and that history does not transfer to a competitor's tool. Switching cost accrues without lock-in tactics.

- **The connector mappings are the integration surface.** Once a client's deploy, flag, campaign and ticket metadata is mapped to blast-radius predicates, that mapping is the asset — and it is the part a rival would have to rebuild client by client.

The architecture itself is publishable, and this document publishes it. The defensibility is in what accumulates on top.

4.9 Delivery shape

This is an engagement, not a product install — which is what makes it a reusable asset rather than a one-off build.

The engine is constant across customers. **The connector mapping is the deliverable**: the declared translation from a client's existing deploy, flag, campaign and ticketing metadata into blast-radius predicates. That is client-specific *configuration*, not custom engineering, and it is the billable work — a 6–8 week shape for a first KPI family, with each additional source a matter of days rather than a rebuild.

Three properties make it land inside an enterprise rather than beside one:

- **No new infrastructure.** The engine is SQL. It executes warehouse-native against the client's existing Snowflake, Databricks or BigQuery, so there is nothing to push through procurement and no second copy of the data to govern.
- **It clears risk and audit on the first pass.** Every number carries a replayable `query_id`, and the two exceptions say so on screen. In practice this — not accuracy — is what decides whether an analytics tool ever reaches production.
- **It fails safely in front of a client.** The system abstains rather than guessing, and names the feed it is missing. A pilot that says *"I cannot explain this yet, connect GitHub"* is a pilot that survives its first surprise.

4.10 How we would know it is working

The feedback loop already collects the product's own success metric: analyst verdicts are rows in a table, so confirmation rate is a query rather than a survey.

Metric	Definition	Target direction
Confirmation rate	analyst 👍 ÷ total verdicts on ranked candidates	up, and *stated* rather than assumed
Time to defensible diagnosis	request → card with evidence	down; the baseline is ~3 analyst-days
Misattribution avoided	candidates rejected by a decisive control that were ranked first on T alone	the decoy case, counted
Abstention rate	diagnoses closing with no candidate above the floor	**non-zero, and monitored as a health metric**

That last row is the one we would watch hardest. A system that never says *"I don't know"* has not become more accurate — it has stopped being honest, and a falling abstention rate is therefore an alarm rather than an improvement.

---

5. Phased roadmap

Shipped — Round 2 prototype

Three KPIs across three sources with different grains; semantic contract; drill-down with contribution; five-component scoring; five negative-control rules; four personas; role entitlement; abstention; the AI investigator lane; the analyst feedback loop; runtime telemetry. **343 tests. All ten Minimum Prototype Expectations close.**

Phase 1 — Rigour (1–2 months)

Item	Why
`effect.py` — difference-in-differences with a bootstrap CI	Turn the observed shortfall into an interval; today no CI is shown because none is computed
Calendar-regressor baseline	Enables **bidirectional** detection; today only drops are flagged, because a quarter-end multiplier would flag every quarter close
`ambiguity.py` — the discriminating test	When two candidates sit within ε, name the one query that separates them
Bounded exploration pass	When nothing clears the floor, spend a capped query budget proposing new slices before abstaining

Phase 2 — Enterprise integration (3–6 months)

Real connectors (GitHub/GitLab, LaunchDarkly, Jira, Zendesk, campaign calendars, pricing tables) replacing synthetic fixtures — *the mapping is the only thing that changes*. Warehouse-native execution against Snowflake, Databricks or BigQuery, since the engine is already just SQL. SSO and per-user verdict attribution. Push delivery into Slack, email and PagerDuty on the channels each persona already declares.

Phase 3 — Proactive (6–12 months)

A watchtower that surfaces anomalies rather than waiting to be pointed; the LLM event normalizer for genuinely unstructured sources, **gated behind the confidence calibration work described in §6**; cross-KPI interaction detection; and a per-tenant learned prior once verdict volume makes it informative.

---

6. Key risks and mitigations

#	Risk	Mitigation	Status
1	**The system names a confident wrong cause**	Negative controls that try to *kill* each candidate; a decisive control failure rejects outright and cannot be outvoted	**Shipped** — the decoy is rejected on camera
2	**The LLM hallucinates a number into the narrative**	Numbers guard: every numeric token must appear in the verified payload; one invented digit discards the narration for the deterministic template	**Shipped**, and the discard is shown on screen
3	**The LLM invents a dimension, metric or check**	Validation against the dimension registry *before* execution; rejections counted and displayed	**Shipped** — "4 accepted, 2 rejected"
4	**The LLM quietly influences the verdict**	Proposed checks built with `decisive=False`, never passed to the scorer; test asserts scores identical with the lane on and off	**Shipped**
5	**Blast radius metadata is wrong or missing**	Failure *direction* is the mitigation: too wide fails its controls, too narrow leaves nothing above the floor. Degrades to "I don't know"	**Shipped**
6	**Precision misread as proof**	The card states outright that it ranks evidence and does not prove causation; the effect is labelled an observed shortfall, not a causal estimate	**Shipped**
7	**Feedback poisons the ranking**	P weighted 0.05 — a prior saturated with 200 confirmations still cannot lift a candidate over the floor or rescue one a control killed; both tested	**Shipped**
8	**Sensitive dimensions leak across roles**	Entitlement enforced at one chokepoint; the card names the policy rather than silently omitting	**Shipped**
9	**Vendor lock-in or model deprecation**	One provider table, one adapter per vendor, one-method interface; two live adapters sharing no code path	**Shipped**
10	**Vendor outage degrades silently**	Every failure path returns empty with a recorded reason; the UI distinguishes "found nothing" from "vendor was down"	**Shipped**
11	**LLM-extracted events outranking recorded ones**	The event normalizer is **deliberately not built** — it is the one design in the original spec that puts a model on the ranking path	**Deferred by decision**, §5 Phase 3
12	**Seasonality mistaken for an incident**	Deseasonalized baseline; the card separates the −0.9% calendar component from the −7.5% that is real	**Shipped**
13	**Verdicts unattributable to a person**	No authentication in this build, so `verdict` records what was decided, not who	**Named, not fixed** — SSO in Phase 2

Appendix A — Minimum Prototype Expectations

#	Expectation	Where it lives
1	3–5 connected KPIs across 2–3 sources, different grains/cadences	`mrr_renewals`, `new_logo_bookings`, `payment_success_rate` (a **ratio** KPI)
2	Lightweight KPI/semantic contract	`contracts_decisions.md` — definitions, calculation SQL, drivers, thresholds, lineage, access
3	≥2 personas with different narratives/actions	Four — `persona_decisions.md`
4	One multi-factor KPI movement with known drivers	Seasonality + deploy + campaign decoy, decomposed on the card
5	One low-confidence scenario: clarification or abstention	Two — sparse KPI declines to detect; source-drop toggle makes refusal reachable (`abstention_decisions.md`)
6	One sparse-history / newly launched KPI	`sparse_kpi_decisions.md`
7	One role-based security scenario	`roles_decisions.md` — `fin.rail_detail`
8	Evidence: freshness, method, contribution, confidence, lineage	Contract panel + evidence chain, every step carrying a `query_id`
9	Clear LLM vs non-LLM breakdown	The ⏱ telemetry panel, on the page — `ai_decisions.md`
10	Runtime telemetry: latency, calls, tokens, cost	Same panel — three query counts, never merged

Ten of ten close.

Appendix B — Round 2 objectives

#	Objective	Status
1	Detects and prioritises material KPI movements	✅ MAD-z + practical-significance gate + BH labelling
2	Reconciles data and business context across heterogeneous sources	✅ Change Ledger; blast radius as dimension predicates
3	Ranks explanatory drivers using appropriate analytical methods	✅ Five-component rubric; every component a query
4	Persona-specific narratives with traceable evidence	✅ Four personas, one computation, identical query ids
5	Communicates uncertainty; abstains when evidence is insufficient	✅ Score floor, abstention branch, connectivity report
6	Actions grounded in levers, constraints and decision rights	✅ The brief's full chain, plus a `query_id` basis
7	Mechanism to learn from analyst and business-user feedback	✅ Beta–Bernoulli prior from analyst verdicts — `learning_decisions.md`
8	Realistic security, cost, latency and scalability constraints	✅ Entitlement; 1.3 s; ~$0.0048/diagnosis; one DuckDB file

Eight of eight.

Appendix C — What is deliberately not built

Naming gaps is the same discipline as abstaining on screen.

- **`ledger/normalizer.py`** — the LLM event normalizer. The one call site in our own design that is *not* additive: it multiplies a hypothesis score by an LLM-emitted confidence, putting a model on the ranking path. Schema and fields stay declared as the record of a deferred decision. (`ai_decisions.md` D2)
- **`effect.py`** — difference-in-differences with bootstrap CI. The impact figure is therefore reported as an *observed shortfall against a deseasonalized baseline*, with no confidence interval, because none is computed. We do not print an interval we did not calculate.
- **`ambiguity.py`** — the discriminating test between near-tied candidates.
- **Bidirectional detection** — v1 flags drops only; the rolling-median baseline does not model the quarter-end multiplier, so a two-sided detector would flag every quarter close. The correct fix is a calendar regressor, not a wider threshold.